Program 10: Create a database and collections and perform CRUD operations using SQL Express.

Aim: To Create a database and collections and perform CRUD operations using SQL Express.

Procedure:

Step 1: Install Required Dependencies

Open the **VS Code Terminal** and install the required Python packages:

pip install django pyodbc

Step 2: Create Django Project & App

Run the following commands:

django-admin startproject employee_project

cd employee_project

django-admin startapp employee_app

Step 3: Configure settings.py for SQL Server

Open employee_project/settings.py and modify the DATABASES section:

```
DATABASES = {
   'default': {
      'ENGINE': 'mssql',
      'NAME': 'YourDatabaseName',
      'USER': 'YourUsername',
      'PASSWORD': 'YourPassword',
      'HOST': 'localhost',
      'PORT': '1433',
      'OPTIONS': {
         'driver': 'ODBC Driver 17 for SQL Server',
      },
   }
}
```

Note: Replace YourDatabaseName, YourUsername, and YourPassword accordingly.

Step 4: Create Django Views (views.py)

Navigate to employee_app/views.py and add the following code for CRUD operations:

Import pyodbc

from django.shortcuts import render, redirect

```
# Function to establish database connection
def get_db_connection():
    conn = pyodbc.connect(
        'DRIVER={ODBC Driver 17 for SQL Server};'
        'SERVER=localhost;'
        'DATABASE=YourDatabaseName;'
        'UID=YourUsername;'
        'PWD=YourPassword;'
    )
    return conn

# Display employee records
def employee_list(request):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT id, name, email, position FROM employee")
    employees = cursor.fetchall()
    conn.close()
    return render(request, 'employee_list.html', {'employees': employees})

# Add Employee
def add_employee(request):
    if request.method == 'POST':
        name = request.POST['name']
        email = request.POST['email']
        position = request.POST['position']

        conn = get_db_connection()
        cursor = conn.cursor()
```

```
        cursor.execute("INSERT INTO employee (name, email, position) VALUES (?, ?, ?)", (name, email, position))
        conn.commit()
        conn.close()
        return redirect('employee_list')

    return render(request, 'add_employee.html')

# Update Employee
def update_employee(request, emp_id):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT id, name, email, position FROM employee WHERE id = ?", (emp_id,))
    employee = cursor.fetchone()
    conn.close()

    if request.method == 'POST':
        name = request.POST['name']
        email = request.POST['email']
        position = request.POST['position']

        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.execute("UPDATE employee SET name = ?, email = ?, position = ? WHERE id = ?", (name, email, position, emp_id))
        conn.commit()
        conn.close()
        return redirect('employee_list')

    return render(request, 'update_employee.html', {'employee': employee})

# Delete Employee
```

```
def delete_employee(request, emp_id):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute("DELETE FROM employee WHERE id = ?", (emp_id,))
    conn.commit()
    conn.close()
    return redirect('employee_list')
```

Step 5: Define URLs (urls.py)

Modify employee_app/urls.py

Create a urls.py file in employee_app/ and add the following:

```
from django.urls import path
from .views import employee_list, add_employee, update_employee, delete_employee

urlpatterns = [
    path('', employee_list, name='employee_list'),
    path('add/', add_employee, name='add_employee'),
    path('update/<int:emp_id>/', update_employee, name='update_employee'),
    path('delete/<int:emp_id>/', delete_employee, name='delete_employee'),
]
```

Modify employee_project/urls.py

Include the app's URL routing:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('employee_app.urls')),
]
```

Step 6: Create HTML Templates

Inside employee_app/templates/, create:

employee_list.html

Displays employee records.

```
<!DOCTYPE html>
<html>
<head>
    <title>Employee List</title>
</head>
<body>
    <h2>Employee List</h2>
    <a href="{% url 'add_employee' %}">Add Employee</a>
    <table border="1">
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Email</th>
            <th>Position</th>
            <th>Actions</th>
        </tr>
        {% for emp in employees %}
        <tr>
            <td>{{ emp.0 }}</td>
            <td>{{ emp.1 }}</td>
            <td>{{ emp.2 }}</td>
            <td>{{ emp.3 }}</td>
            <td>
                <a href="{% url 'update_employee' emp.0 %}">Edit</a> |
                <a href="{% url 'delete_employee' emp.0 %}">Delete</a>
            </td>
        </tr>
        {% endfor %}
    </table>
</body>
```

```
</html>
```

update_employee.html

Form for updating an employee.

```
<!DOCTYPE html>
<html>
<head>
  <title>Update Employee</title>
</head>
<body>
  <h2>Update Employee</h2>
  <form method="post">
    {% csrf_token %}
    <label>Name:</label> <input type="text" name="name" value="{{ employee.1 }}" required><br>
    <label>Email:</label> <input type="email" name="email" value="{{ employee.2 }}"
required><br>
    <label>Position:</label> <input type="text" name="position" value="{{ employee.3 }}"
required><br>
    <button type="submit">Update</button>
  </form>
  <a href="{% url 'employee_list' %}">Back to Employee List</a>
</body>
</html>
```

Step 7: Run the Django Server

python manage.py runserver

Now open **http://127.0.0.1:8000/** in a web browser.

OutPut:

Employee List

Add Employee

ID	Name	Email	Position	Actions
4	ram	ram@123	tester	Edit | Delete
5	asd	asd@gmail.com	tester	Edit | Delete

Update Employee

Name: ram
Email: ram@123
Position: tester2
Update
Back to Employee List